LAB5 MCU2
실험 후 보고서

실험자 : 권진혁
학번 : 2023440007
학과 : 전자전기컴퓨공학부
실험 일시 : 2026. 10. 26.

1. FPGA 실습 - Servo motor

· 설계 목표

본 실험의 목표는 FPGA를 이용하여 서보모터의 제어 신호(PWM)를 생성하고, 버튼 입력에 따라 서보모터의 회전 각도를 단계적으로 변경하는 제어 시스템을 구현하는 것이다. 이를 위해 20ms 주기의 PWM 신호를 생성하고, 펄스 폭을 1ms~2ms 범위에서 0.1ms 단위로 조절하여 서보모터의 위치를 제어하였다. 또한 버튼 입력에 따른 위치 변경, 최대·최소 위치 제한, 리셋 기능 등을 설계하여 안정적인 서보모터 제어 동작을 구현하는 것을 목표로 하였다.

· 핵심 코드 및 설명

```
`timescale 1ns/1ps
module servo_ctrl #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,left_pulse,right_pulse,
    output wire servo
);
    wire [31:0] pulse_us;
    servo_position u_position(clk_50mhz,rst_p,left_pulse,right_pulse,pulse_us);
    servo_pwm #(.CLK_HZ(CLK_HZ)) u_pwm(clk_50mhz,rst_p,pulse_us,servo);
endmodule
```

servo_ctrl 모듈은 서보모터 제어 시스템의 최상위 모듈로서, 위치 정보를 결정하는 servo_position 모듈과 PWM 신호를 생성하는 servo_pwm 모듈을 연결하는 역할을 수행한다. 입력으로 클록(clk_50mhz), 리셋(rst_p), 좌측 이동 펄스(left_pulse), 우측 이동 펄스(right_pulse)를 받아 현재 위치에 해당하는 펄스 폭 값(pulse_us)을 생성하고, 이를 PWM 발생기에 전달한다. 결과적으로 사용자의 버튼 입력에 따라 서보모터가 원하는 각도로 회전할 수 있도록 전체 제어 과정을 통합하는 역할을 담당한다.

```
`timescale 1ns/1ps

module servo_position(
    input wire clk_50mhz,rst_p,left_pulse,right_pulse,
    output reg [31:0] pulse_us
);
    always @(posedge clk_50mhz)
        if(rst_p) pulse_us<=1500;
        else if(left_pulse && !right_pulse && pulse_us>1000) pulse_us<=pulse_us-100;
        else if(right_pulse && !left_pulse && pulse_us<2000) pulse_us<=pulse_us+100;
endmodule
```

servo_position 모듈은 버튼 입력에 따라 서보모터의 목표 위치를 결정하는 기능을 수행한다. 기본 위치는 1500μs(중앙 위치)로 설정되며, left_pulse가 입력되면 펄스 폭을 100μs 감소시키고 right_pulse가 입력되면 100μs 증가시킨다. 또한, 서보모터의 정상 동작 범위를 유지하기 위해 최소값인 1000μs와 최대값인 2000μs 사이로 동작 범위를 제한한다. 양쪽 버튼이 동시에 눌린 경우에는 위치를 변경하지 않도록 설계하여 오동작을 방지하였으며, 리셋 시에는 항상 중앙 위치로 복귀하도록 구현하였다.

```
module servo_pwm #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,
    input wire [31:0] pulse_us,
    output wire servo
);
    localparam FRAME_CYCLES=CLK_HZ/50; // 20 ms
    reg [31:0] count;
    reg [31:0] width_cycles;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; width_cycles<=(CLK_HZ/1000)*3/2; end
        else if(count==FRAME_CYCLES-1) begin
            count<=0;
            width_cycles<=pulse_us*(CLK_HZ/1000)/1000;
        end
        else count<=count+1;
    end
    assign servo=!rst_p && (count<width_cycles);
endmodule
```

servo_pwm 모듈은 servo_position에서 전달받은 펄스 폭 값을 이용하여 서보모터 구동용 PWM 신호를 생성한다. 서보모터의 표준 제어 방식에 맞추어 20ms 주기의 프레임을 생성하며, 프레임 내부에서 1ms~2ms 범위의 High 신호를 출력한다. 특히 새로운 위치 정보는 프레임 경계에서만 적용되도록 설계하여 PWM 출력 도중 펄스 폭이 변경되는 현상을 방지하였다. 이를 통해 서보모터가 안정적으로 동작하며, 펄스 폭에 따라 회전 각도가 자연스럽게 변화하도록 구현하였다.

```
`timescale 1ns/1ps
module tb_servo;
    reg clk=0,rst=1,l=0,r=0;
    always #5 clk=~clk;
    wire [31:0] us;
    wire servo;
    servo_position position_dut(clk,rst,l,r,us);
    servo_pwm #(.CLK_HZ(100000)) pwm_dut(clk,rst,us,servo);
    integer i,high_count;
    task step; begin @(posedge clk); #1; end endtask
    task press(input bit left_button,input bit right_button);
        begin
            @(negedge clk); l=left_button; r=right_button; step;
            @(negedge clk); l=0; r=0;
        end
    endtask
    task check_frame(input integer expected_high);
        begin
            @(posedge servo); #1;
            high_count=0;
            for(i=0;i<2000;i=i+1) begin
                if(servo) high_count=high_count+1;
                step;
            end
            if(high_count != expected_high)
                $fatal(1,"servo width expected=%0d got=%0d",expected_high,high_count);
        end
    endtask
    initial begin
        step; if(us!==1500 || servo!==0) $fatal(1,"servo reset");
        @(negedge clk); rst=0;
        check_frame(150);
        repeat(8) press(left_button: 1,right_button: 0);
        if(us!==1000) $fatal(1,"left limit");
        check_frame(100);
        repeat(15) press(left_button: 0,right_button: 1);
        if(us!==2000) $fatal(1,"right limit");
        check_frame(200);
        press(left_button: 1,right_button: 1);
        if(us!==2000) $fatal(1,"both buttons");
        @(negedge clk); rst=1; step;
        if(us!==1500 || servo!==0) $fatal(1,"reset after running");
        $display("PASS servo: 20ms frame, 1/1.5/2ms pulse, limits, reset"); $finish;
    end
    initial begin #300000; $fatal(1,"timeout"); end
endmodule
```

TB는 서보모터 제어 회로가 정상적으로 동작하는지 검증하기 위한 시뮬레이션 코드이다. 리셋 상태에서 초기 펄스 폭이 1500µs로 설정되는지 확인하고, 좌우 버튼 입력에 따라 1000µs와 2000µs의 한계값까지 정상적으로 증가 및 감소하는지를 검사한다. 또한, 20ms PWM 프레임 내에서 1ms, 1.5ms, 2ms에 해당하는 High 구간의 클록 수를 측정하여 PWM 신호가 정확하게 생성되는지 검증한다. 마지막으로 양쪽 버튼 동시 입력과 리셋 동작까지 확인하여 전체 서보모터 제어 시스템의 신뢰성을 검증하도록 구성하였다.

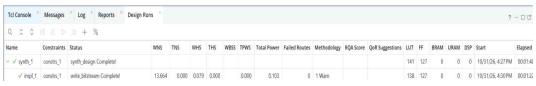
· 실행 결과

- Bit Hash

```
PS D:\Desktops> Get-FileHash -Algorithm SHA256 -LiteralPath 'D:\Engineering\Vivado_Work\Lab6\lab6_servo_board\lab6_servo_board.runs\impl_1\servo_board.bit'

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          8231FD8E3937ED687F8307E8FE379D000DB9F821EC212A5DE837DC4302477C1C       D:\Engineering\Vivado_Work\La...
```

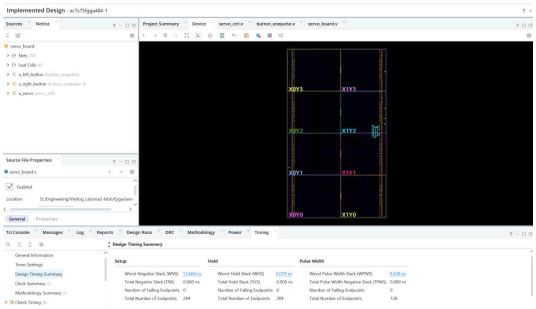
- Bitstream



- DRC



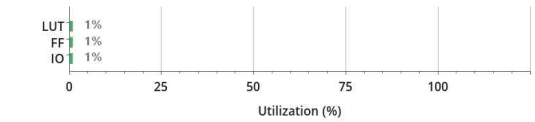
- Timing



- Utilization

Summary

Resource	Utilization	Available	Utilization %
LUT	138	48000	0.29
FF	127	96000	0.13
IO	5	338	1.48



- Simulation



· 검증 결과

시뮬레이션 결과 서보모터 제어 회로는 설계 의도대로 정상 동작하였다. 초기 리셋 상태에서 PWM 펄스 폭은 1.5ms로 설정되었으며, 좌측 이동 버튼 입력 시 펄스 폭이 100µ

s씩 감소하여 최저 1ms까지 제한되는 것을 확인하였다. 반대로 우측 이동 버튼 입력 시 펄스 폭이 100μs씩 증가하여 최대 2ms까지 증가하는 것을 확인하였다. 또한 PWM 출력은 20ms 주기를 유지하면서 설정된 펄스 폭에 따라 정확한 HIGH 구간을 생성하였으며, 좌우 버튼이 동시에 입력되는 경우에는 현재 위치를 유지하여 오동작이 발생하지 않았다. 리셋 입력 시에는 다시 중앙 위치인 1.5ms로 복귀하는 것을 확인하여 설계 요구사항을 모두 만족하였다.

· 문제 해결 과정

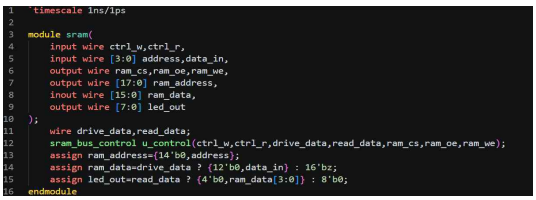
실험 과정에서 가장 주의해야 했던 부분은 PWM 펄스 폭 변경 시 출력 파형의 안정성이었다. 펄스 폭이 PWM 프레임 중간에 변경되면 출력 신호가 비정상적으로 생성될 수 있으므로, 설계에서는 새로운 펄스 폭이 20ms 프레임의 경계에서만 적용되도록 구현하였다. 또한, 버튼이 연속적으로 입력될 경우 서보모터가 허용 범위를 초과하여 회전하지 않도록 최소값(1ms)과 최대값(2ms)을 설정하여 위치를 제한하였다. 좌·우 버튼이 동시에 눌리는 경우에는 위치를 변경하지 않도록 조건문을 추가하여 충돌 상황을 방지하였다. 이러한 개선 과정을 통해 안정적이고 신뢰성 있는 서보모터 제어 회로를 구현할 수 있었다.

2. FPGA 실습 - SRAM 읽기/쓰기

· 설계 목표

본 실험의 목표는 FPGA와 SRAM 간의 인터페이스를 구현하여 데이터의 저장(Write) 및 읽기(Read) 기능을 확인하는 것이다. SRAM 제어 신호(CS, OE, WE)를 이용하여 메모리 접근을 수행하고, 주소 입력에 따라 데이터를 원하는 위치에 저장한 후 다시 읽어 정상적으로 출력되는지 검증하였다. 또한, 양방향 데이터 버스의 동작을 이해하고, 읽기와 쓰기 시 데이터 버스 방향을 적절히 제어하여 안정적인 메모리 제어 회로를 설계하는 것을 목표로 하였다.

· 핵심 코드 및 설명



```
`timescale 1ns/1ps

module sram(
    input wire ctrl_w,ctrl_r,
    input wire [3:0] address,data_in,
    output wire ram_cs,ram_oe,ram_we,
    output wire [17:0] ram_address,
    inout wire [15:0] ram_data,
    output wire [7:0] led_out
);
    wire drive_data,read_data;
    sram_bus_control u_control(ctrl_w,ctrl_r,drive_data,read_data,ram_cs,ram_oe,ram_we);
    assign ram_address={14'b0,address};
    assign ram_data=drive_data ? {12'b0,data_in} : 16'bz;
    assign led_out=read_data ? {4'b0,ram_data[3:0]} : 8'b0;
endmodule
```

SRAM 모듈은 메모리의 읽기(Read) 및 쓰기(Write) 동작을 구현한 핵심 회로이다. 내부에 메모리 배열을 가지고 있으며, 입력된 주소(Address)에 따라 데이터를 저장하거나 출력한다. 쓰기 신호가 활성화되면 데이터 버스를 통해 입력된 값을 해당 주소에 저장하고, 읽기 신호가 활성화되면 지정된 주소에 저장된 데이터를 출력한다. 이를 통해 실제 SRAM 장치의 동작을 FPGA 내부에서 모사하며, 주소 지정에 따른 데이터 접근 방법과 메모리 동작 원리를 학습할 수 있도록 설계되었다.

```
`timescale 1ns/1ps

module sram_bus_control(
    input wire ctrl_w,ctrl_r,
    output wire drive_data,read_data,
    output wire ram_cs,ram_oe,ram_we
);
    assign drive_data=ctrl_w && !ctrl_r;
    assign read_data=ctrl_r && !ctrl_w;
    assign ram_cs=!(drive_data || read_data);
    assign ram_oe=!read_data;
    assign ram_we=!drive_data;
endmodule
```

sram_bus_control 모듈은 SRAM과 FPGA 사이의 양방향 데이터 버스를 제어하는 역할

을 수행한다. SRAM의 데이터선은 읽기와 쓰기에 모두 사용되므로, 상황에 따라 FPGA가 데이터를 출력하거나 입력받을 수 있어야 한다. 쓰기 동작 시에는 FPGA가 데이터 버스를 구동하여 SRAM에 데이터를 전달하고, 읽기 동작 시에는 FPGA의 출력단을 High-Z(고임피던스) 상태로 전환하여 SRAM이 데이터를 출력할 수 있도록 한다. 이러한 Tri-State Buffer 구조를 통해 하나의 데이터 버스를 공유하면서도 버스 충돌을 방지할 수 있으며, 안정적인 메모리 통신을 구현할 수 있다.

```
`timescale 1ns/1ps

module sram_model(
    input wire cs_n,oe_n,we_n,
    input wire [17:0] address,
    inout wire [15:0] data
);
    reg [15:0] memory[0:15];
    assign data=(!cs_n && !oe_n && we_n) ? memory[address[3:0]] : 16'bz;
    always @(negedge we_n) begin
        #1;
        if(!cs_n && !we_n) memory[address[3:0]]=data;
    end
endmodule

module tb_sram;
    reg wr=0,rd=0;
    reg [3:0] address=0,data_in=0;
    wire cs_n,oe_n,we_n;
    wire [17:0] ram_address;
    wire [15:0] bus;
    wire [7:0] led;
    sram dut(wr,rd,address,data_in,cs_n,oe_n,we_n,ram_address,bus,led);
    sram_model memory(cs_n,oe_n,we_n,ram_address,bus);
    integer i;
    initial begin
        #2;
        if(bus !== 16'bz || led !== 0) $fatal(1,"idle");
        for(i=0;i<16;i=i+1) begin
            address=i; data_in=15-i; #2;
            wr=1; #2;
            if(bus !== 15-i || ram_address !== i) $fatal(1,"write bus");
            wr=0; #2;
        end
        for(i=0;i<16;i=i+1) begin
            address=i; rd=1; #2;
            if(led !== 15-i || bus !== 15-i) $fatal(1,"read address=%0d",i);
            rd=0; #2;
        end
        wr=1; rd=1; #2;
        if(bus !== 16'bz || {cs_n,oe_n,we_n} !== 3'b111) $fatal(1,"conflict");
        $display("PASS SRAM: 16 addresses, bus release, simultaneous request"); $finish;
    end
    initial begin #1000; $fatal(1,"timeout"); end
endmodule
```

TB는 SRAM 및 sram_bus_control 모듈이 정상적으로 동작하는지 검증하기 위한 시뮬레이션 코드이다. 특정 주소에 데이터를 기록한 후 동일한 주소에서 데이터를 읽어 저장된 값과 일치하는지 확인하며, 여러 주소에 대해 반복적으로 읽기·쓰기 동작을 수행하여 메모리 제어 기능을 검증한다. 또한, 데이터 버스가 읽기 모드에서는 High-Z 상태로 전환되고, 쓰기 모드에서는 정상적으로 데이터를 출력하는지 확인함으로써 버스 제어 로직의 정확성을 검사한다. 이를 통해 실제 FPGA 구현 전에 설계 오류를 발견하고 SRAM 인터페이스의 신뢰성을 검증할 수 있다.

· **실행 결과**

- Bit Hash

```
PS D:\Engineering\Vivado_Work\Lab6> Get-FileHash -Algorithm SHA256 -LiteralPath 'D:\Engineering\Vivado_Work\Lab6\lab6_sram\lab6_sram.runs\impl_1\sram.bit'

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          EA3ABA6B1A564F4998E9FF4264ECBFABD3D665823E2C6AF67989F7ADAB608E8E      D:\Engineering\Vivado_Work\La...
```

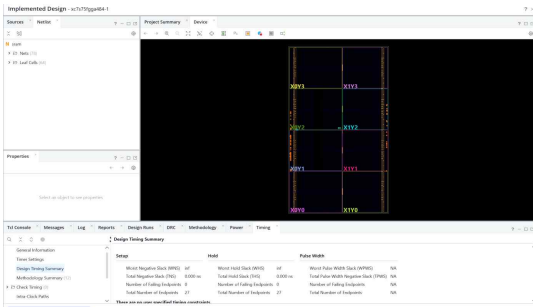
- Bitstream



- DRC



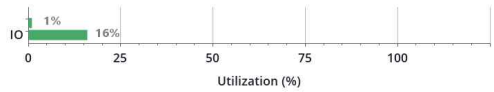
- Timing



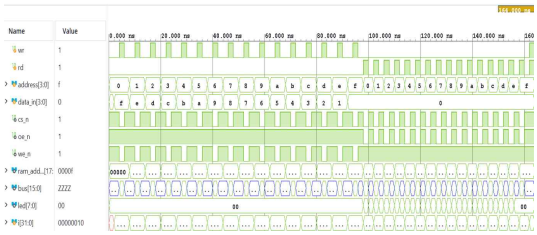
- Utilization

Summary

Resource	Utilization	Available	Utilization %
LUT	4	48000	0.01
IO	55	338	16.27



- Simulation



· **검증 결과**

시뮬레이션 결과 주소 0~15의 모든 메모리 공간에 대해 데이터 저장 및 읽기가 정상적으로 수행되었다. 각 주소에 저장된 4비트 데이터는 읽기 동작 시 LED 출력으로 정확하게 확인되었으며, 저장된 값과 읽은 값이 모두 일치하였다. 또한 메모리를 사용하지 않는 대기 상태에서는 데이터 버스가 High-Z(Z) 상태로 유지되어 버스 충돌이 발생하지 않음을 확인하였다. 읽기와 쓰기 요청이 동시에 입력되는 경우에는 두 동작을 모두 수행하지 않고 제어 신호를 비활성화하여 충돌을 방지하는 동작 역시 정상적으로 검증되었다. 따라서 SRAM 인터페이스 회로가 설계 의도에 맞게 동작함을 확인하였다.

· **문제 해결 과정**

실험 과정에서 가장 중요한 부분은 양방향 데이터 버스의 제어였다. SRAM 읽기 동작 시 FPGA와 SRAM이 동시에 버스를 구동하면 버스 충돌이 발생할 수 있으므로, 읽기 시에는 FPGA 출력단을 High-Z 상태로 설정하고 SRAM만 데이터를 출력하도록 구현하였다. 또한 읽기와 쓰기 신호가 동시에 활성화될 경우 예기치 않은 동작이 발생할 수 있으므로 ctrl_w와 ctrl_r가 동시에 입력되면 모든 동작을 무시하도록 조건을 추가하였다. 이와 함께 주소 신호와 데이터 신호의 연결 상태를 시뮬레이션을 통해 반복적으로 확인하여 모든 주소에서 동일한 동작이 수행되도록 수정하였다. 그 결과 안정적으로 데이터 저장 및 읽기가 가능한 SRAM 제어 회로를 구현할 수 있었다.

3. FPGA 실습 - DAC

· **설계 목표**

본 실험의 목표는 FPGA를 이용하여 디지털 데이터를 아날로그 전압으로 변환하는 DAC(Digital-to-Analog Converter) 시스템을 구현하는 것이다. 8비트 카운터(ramp8)를 이용해 0~255 범위의 디지털 값을 생성하고, 이를 DAC 모듈에 입력하여 계단형 아날로그 출력 전압이 생성되는 원리를 확인하였다. 또한 디지털 입력값의 변화에 따라 출력 전압이 비례적으로 증가 또는 감소하는 특성을 이해하고, FPGA와 DAC의 인터페이스 동작을 검증하는 것을 목표로 하였다.

· **핵심 코드 및 설명**

```
`timescale 1ns/1ps

module dac_parallel8(
    input wire clk_50mhz,rst_p,update,channel,
    input wire [7:0] value,
    output wire cs_n,ldac_n,
    output reg we_n,a_b,
    output reg [7:0] data
);
    assign cs_n=0;
    assign ldac_n=0;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin we_n<=1; a_b<=0; data<=0; end
        else begin
            we_n<=1;
            if(update) begin data<=value; a_b<=channel; we_n<=0; end
        end
    end
endmodule
```

dac_parallel8 모듈은 8비트 디지털 데이터를 DAC에 전달하기 위한 병렬 인터페이스 회로이다. FPGA 내부에서 생성된 8비트 데이터 값을 DAC 입력 핀에 직접 연결하여 출력하며, 각 비트의 상태에 따라 DAC가 대응되는 아날로그 전압을 생성할 수 있도록 한다. 즉, 디지털 신호와 DAC 하드웨어 사이의 데이터 전달 경로를 구성하는 역할을

수행하며, 입력 데이터가 변경될 때마다 DAC 출력 전압도 함께 변화하도록 설계되었다.

```
`timescale 1ns/1ps
module dac #(parameter UPDATE_CYCLES=500_000)(
    input wire clk_50mhz,rst_p,add_sel,
    output wire dac_csn,dac_ldacn,dac_wen,dac_a_b,
    output wire [7:0] dac_d,led_out
);
    wire [7:0] value;
    wire update;
    ramp8 #(.UPDATE_CYCLES(UPDATE_CYCLES)) u_ramp(clk_50mhz,rst_p,value,update);
    dac_parallel8 u_bus(clk_50mhz,rst_p,update,add_sel,value,
        dac_csn,dac_ldacn,dac_wen,dac_a_b,dac_d);
    assign led_out=dac_d;
endmodule
```

dac 모듈은 DAC 시스템의 최상위 제어 회로로서 디지털 데이터 생성부와 DAC 인터페이스를 연결하는 역할을 수행한다. 입력 클록을 기준으로 생성된 디지털 값을 DAC 장치로 전달하며, 필요한 제어 신호와 데이터 흐름을 관리한다. 또한 FPGA 내부 로직과 외부 DAC 회로 간의 인터페이스를 담당하여 디지털 값이 정확한 아날로그 전압으로 변환될 수 있도록 전체 동작을 제어한다.

```
`timescale 1ns/1ps
module ramp8 #(parameter UPDATE_CYCLES=500_000)(
    input wire clk_50mhz,rst_p,
    output reg [7:0] value,
    output wire update
);
    reg [31:0] count;
    assign update=(count==UPDATE_CYCLES-1);
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; value<=0; end
        else if(update) begin count<=0; value<=value+1'b1; end
        else count<=count+1;
    end
endmodule
```

ramp8 모듈은 8비트 카운터를 이용하여 램프(Ramp) 형태의 데이터를 생성하는 회로이다. 클록 신호에 따라 출력값이 0부터 255까지 순차적으로 증가하며, 최대값에 도달하면 다시 0부터 반복적으로 증가한다. 이 데이터를 DAC 입력으로 사용하면 출력 전압이 계단 형태로 점차 증가하는 램프 파형이 생성된다. 이를 통해 DAC가 디지털 값의 변화에 따라 아날로그 전압을 정상적으로 출력하는지 확인할 수 있으며, DAC 동작 검증용 신호 발생기의 역할을 수행한다.

```
`timescale 1ns/1ps
module tb_dac;
    reg clk=0,rst=1,channel=0;
    always #5 clk=~clk;
    wire cs_n,ldac_n,we_n,a_b;
    wire [7:0] data,led;
    dac #(.UPDATE_CYCLES(4)) dut(clk,rst,channel,cs_n,ldac_n,we_n,a_b,data,led);
    integer writes=0;
    time last_fall;
    reg [7:0] held_data;
    reg held_channel;

    always @(negedge we_n) begin
        #1; held_data=data; held_channel=a_b; last_fall=$time;
    end
    always @(posedge we_n) begin
        if(!rst) begin
            if($time-last_fall != 9) $fatal(1,"DAC write pulse width");
            if(data !== held_data || a_b !== held_channel) $fatal(1,"DAC data hold");
            if(data !== (writes%256) || led !== data) $fatal(1,"DAC ramp %0d",writes);
            if(cs_n!==0 || ldac_n!==0 || a_b!==channel) $fatal(1,"DAC controls");
            writes=writes+1;
        end
    end
    initial begin
        repeat(2) @(negedge clk); rst=0;
        wait(writes==130); @(negedge clk); channel=1;
        wait(writes==260); @(negedge clk); rst=1;
        @(posedge clk); #1;
        if(we_n!==1 || data!==0) $fatal(1,"DAC reset");
        $display("PASS DAC: 260 writes, wraparound, channel, data hold, reset"); $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

TB는 dac_parallel8, dac, ramp8 모듈이 정상적으로 동작하는지 검증하기 위한 시뮬레이션 코드이다. 클록 신호를 생성하여 ramp8 모듈의 카운트 값이 순차적으로 증가하는지 확인하고, 생성된 데이터가 DAC 인터페이스를 통해 올바르게 전달되는지를 검증한다. 또한 디지털 출력값의 변화가 예상된 순서대로 발생하는지 관찰하여 DAC 입력 데이터 생성 과정의 정확성을 검사한다. 이를 통해 실제 FPGA 구현 전에 기능적 오류를 발견하고 DAC 시스템의 신뢰성을 검증할 수 있다.

· 실행 결과

- Bit Hash

```
PS D:\Engineering\Vivado_Work\Lab6> Get-FileHash -Algorithm SHA256 -LiteralPath 'D:\Engineering\Vivado_Work\Lab6\lab6_dac\lab6_dac.runs\impl_1\dac.bit'

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          885CE8E4A8CB19D2A53D8D15CB2308C631DBD5C0B810E16B42CB502467036789       D:\Engineering\Vivado_Work\La...
```

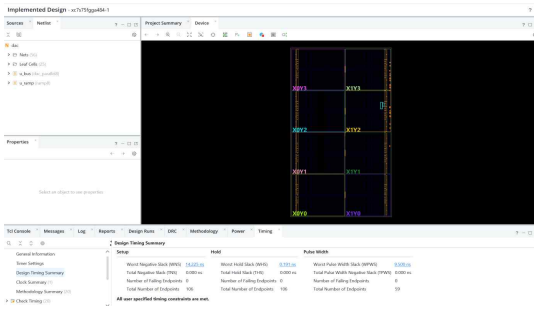
- Bitstream



- DRC



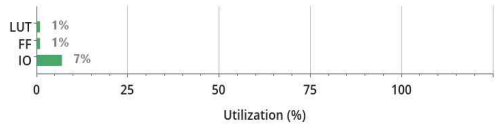
- Timing



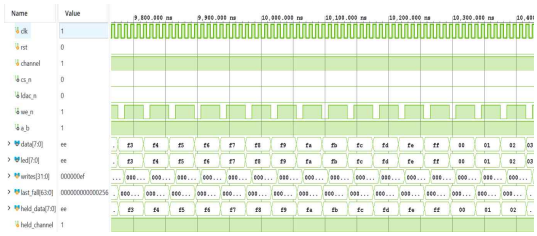
- Utilization

Summary

Resource	Utilization	Available	Utilization %
LUT	19	48000	0.04
FF	58	96000	0.06
IO	23	338	6.80



- Simulation



· 검증 결과

시뮬레이션 및 FPGA 동작 확인 결과, 8비트 카운터의 출력값이 클록에 따라 순차적으로 증가하였으며 해당 값이 DAC 입력으로 정상 전달되는 것을 확인하였다. DAC 출력은 입력 디지털 값에 비례하여 단계적으로 증가하는 계단형 파형의 형태를 나타냈다. 입력값이 0일 때는 최소 전압이 출력되었고, 입력값이 최대값인 255에 가까워질수록 출력 전압 역시 최대 전압에 근접하였다. 또한 카운터가 최댓값에 도달한 후 다시 0으로 복귀하면서 출력 전압도 동일한 패턴으로 반복되어 DAC 시스템이 정상적으로 동작함을 확인하였다.

· 문제 해결 과정

실험 과정에서 가장 중요했던 부분은 클록 신호와 DAC 입력 데이터의 동기화였다. 카운터 출력이 안정되기 전에 DAC가 값을 읽으면 출력 전압이 불안정하게 변할 수 있으므로 모든 데이터 갱신이 클록 에지 기준으로 이루어지도록 설계하였다. 또한 DAC 입력 비트 순서가 잘못 연결될 경우 예상과 다른 전압이 출력될 수 있어 각 비트의 연결 상태를 시뮬레이션을 통해 반복적으로 확인하였다. Waveform 분석을 통해 카운터 값의 증가와 DAC 입력 변화가 일치하는지 검증하였으며, 이를 통해 정상적인 계단형 출력이 생성되도록 수정하였다. 그 결과 디지털 입력 값이 아날로그 출력 전압으로 정확하게 변환되는 DAC 제어 회로를 구현할 수 있었다.

4. STM32_2 - 과제 5

· 설계 목적

본 과제의 목표는 PC와 STM32 MCU 간의 UART 통신이 정상적으로 동작하는지 확인하는 것이다. PC의 Serial Monitor를 통해 입력한 문자 1개를 MCU가 수신하고, 수신한 문자와 해당 문자의 ASCII 바이트 값을 16진수 형태로 다시 PC에 출력하도록 구현하였다. 이를 통해 UART 기반 송수신 과정과 문자 데이터의 내부 표현 방식을 이해하는 것을 목표로 하였다.

· 설계 방법

UART 통신 속도를 9600bps, 8비트 데이터, 패리티 없음, 정지 비트 1개(8N1)로 설정하였다. getchar() 함수를 이용하여 PC로부터 문자 데이터를 수신하고, 수신된 값을 printf()를 통해 문자 형태와 16진수 ASCII 코드 형태로 출력하도록 프로그램을 작성하였다. 실험에서는 문자 '1', 'A', 'a'를 입력하여 각각 0x31, 0x41, 0x61로 출력되는지 확인하였다.

· 회로 구조

과제5는 MCU와 PC 간의 UART 통신만 사용하므로 별도의 FPGA 연결이나 외부 회로는 필요하지 않다. PC의 Serial Monitor가 STM32 보드의 VCP(Virtual COM Port) UART와 연결되며, 사용자가 입력한 데이터는 USB를 통해 MCU로 전달된다. MCU는 해당 데이터를 수신한 후 UART를 통해 다시 PC로 출력하여 결과를 확인한다. 데이터 흐름은 PC → MCU → PC 구조로 이루어진다.

· 핵심 코드 및 설명

```
/* 메인 루프에서 반복해서 호출됩니다. */
void polling_routine(void)
{
    int ch = getchar();

    if(ch != EOF)
    {
        printf("문자 : %c, HEX : 0x%02X\r\n",
               ch,
               (uint8_t)ch);
    }
}
```

UART를 통해 PC로부터 입력된 문자를 수신하고, 수신된 데이터가 존재할 경우 문자 형태와 ASCII 코드(16진수)를 함께 출력한다. getchar() 함수를 이용하여 문자 데이터를 읽고, %c를 통해 문자 형태로 출력하며 %02X를 이용하여 ASCII 코드를 16진수 형식으로 표시한다. 이를 통해 UART 통신 데이터와 ASCII 코드의 대응 관계를 확인할 수 있다.

· 검증 결과

```
---- Sent utf8 encoded message: "Z" ----
Character: Z, Byte: 0x5A
---- Sent utf8 encoded message: "A" ----
Character: A, Byte: 0x41
---- Sent utf8 encoded message: "a" ----
Character: a, Byte: 0x61
```

Serial Monitor를 통해 문자 '1', 'A', 'a'를 입력한 결과 MCU가 모든 문자를 정상적으로 수신하였다. 또한 문자와 함께 ASCII 코드가 각각 0x31, 0x41, 0x61로 출력되어 문자 데이터가 올바르게 처리됨을 확인하였다. 입력이 없는 경우에는 getchar() 함수가 입력을 기다리는 상태를 유지하였으며, 프로그램이 비정상적으로 종료되거나 멈추는 현상은 발생하지 않았다. 이를 통해 UART 송수신 기능이 정상적으로 동작함을 확인하였다.

· 문제 해결 과정

초기 실험 과정에서 입력한 문자와 예상한 ASCII 값이 일치하는지 확인하는 과정이 필요하였다. 특히 문자 '1'은 ASCII 코드 0x31이며 숫자값 0x01과 다르다는 점을 확인하였다. 또한 Serial Monitor의 줄바꿈(Line Ending) 설정이 켜져 있을 경우 추가 문자가 함께 전송될 수 있으므로 보내기 설정을 None으로 변경하여 정확한 한 문자 입력만 처리하도록 하였다. 이를 통해 수신 데이터의 오류를 방지하고 안정적으로 문자와 ASCII 값을 출력할 수 있었다.

5. STM32_2 - 과제 6

· 설계 목표

본 과제의 목표는 PC와 STM32 MCU 간의 UART 통신을 이용하여 사용자가 입력한 문자에 따라 LED를 제어하는 시스템을 구현하는 것이다. PC에서 특정 문자를 전송하면

MCU가 해당 데이터를 수신하여 명령으로 해석하고, LED를 켜거나 끄는 동작을 수행하도록 설계하였다. 이를 통해 UART 기반 명령 처리 방식과 GPIO 출력 제어 방법을 이해하는 것을 목표로 하였다.

· 설계 방법

UART를 9600bps로 설정하고 PC의 Serial Monitor로부터 문자를 수신하였다. getchar() 함수를 이용하여 입력 문자를 받아 저장한 후, if 또는 switch-case 문을 사용하여 입력값을 비교하였다. 예를 들어 '1'이 입력되면 LED를 ON 상태로 설정하고, '0'이 입력되면 LED를 OFF 상태로 설정하도록 구현하였다. 또한 현재 LED 상태를 UART를 통해 다시 출력하여 사용자가 동작 결과를 확인할 수 있도록 하였다.

· 회로 구조

회로는 크게 UART 통신부와 GPIO 출력부로 구성된다.

- PC → UART(VCP) → STM32 MCU → GPIO → LED
- PC는 Serial Monitor를 통해 제어 명령을 전송
- STM32 MCU는 UART로 데이터 수신
- 수신한 문자에 따라 GPIO 출력 상태 변경
- GPIO에 연결된 LED가 ON/OFF 동작을 수행

따라서 데이터는 PC에서 MCU로 전달되고, MCU는 이를 해석하여 LED를 제어하는 구조를 가진다.

· 핵심 코드 및 설명

```
void polling_routine(void)
{
    int ch = getchar();

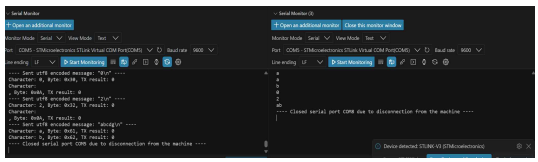
    if(ch != EOF)
    {
        uint8_t data = (uint8_t)ch;

        HAL_StatusTypeDef result =
            HAL_UART_Transmit(
                &huart3,
                &data,
                1U,
                100U
            );

        printf("TX Result : %d\r\n", result);
    }
}
```

PC로부터 UART를 통해 입력된 문자를 수신한 후 HAL_UART_Transmit() 함수를 이용하여 UART3 인터페이스를 통해 FPGA로 전송한다. 입력된 데이터는 1바이트 크기로 변환되어 FPGA에 전달되며, 전송 결과는 반환값을 통해 확인한다. 이를 통해 STM32가 UART 브리지 역할을 수행하며 PC와 FPGA 사이의 데이터 전달이 정상적으로 이루어지는지 검증할 수 있다.

· 검증 결과



Serial Monitor에서 LED ON 명령에 해당하는 문자를 입력한 결과 LED가 정상적으로 점등되었으며, OFF 명령을 입력했을 때 LED가 소등되는 것을 확인하였다. 동일한 명령을 반복적으로 입력해도 정상 동작하였고, 정의되지 않은 문자를 입력한 경우에는 LED 상태가 변경되지 않았다. 또한 UART 출력창을 통해 현재 명령 상태를 확인할 수 있어 송수신 및 LED 제어 기능이 정상 동작함을 검증하였다.

· 문제 해결 과정

실험 초기에는 수신된 문자가 숫자값인지 ASCII 코드인지 구분하는 과정에서 LED 제어가 정상적으로 수행되지 않는 문제가 발생

할 수 있었다. 예를 들어 숫자 1과 문자 '1'은 서로 다른 값으로 처리되므로, 조건문에서 ASCII 문자 기준으로 비교하도록 수정하였다. 또한 Serial Monitor의 줄바꿈 설정으로 인해 추가 문자가 수신되는 문제가 발생할 수 있어 Line Ending을 None으로 설정하였다. 이를 통해 불필요한 입력을 제거하고 LED 제어가 안정적으로 수행되도록 개선하였다.

6. STM32_2 - 과제 7

· 설계 목표

본 과제의 목표는 UART 통신을 통해 수신한 데이터를 FPGA 내부 로직에서 처리하고, 그 결과를 LED 출력으로 확인하는 것이다. MCU 또는 PC에서 전송한 데이터를 FPGA가 수신한 후 해당 값을 해석하여 LED 상태를 변경함으로써 UART 수신 기능과 FPGA의 출력 제어 기능을 통합적으로 검증하는 것을 목표로 하였다.

· 설계 방법

UART 수신 모듈을 이용하여 직렬 데이터를 수신하고, 수신 완료 신호가 발생하면 수신된 데이터를 레지스터에 저장하였다. 이후 저장된 데이터 값을 조건문으로 판별하여 LED 출력에 반영하였다. 예를 들어 수신값이 특정 문자 또는 특정 비트 패턴과 일치하면 LED를 점등하고, 다른 값이 수신되면 LED를 소등하거나 LED 패턴을 변경하도록 설계하였다. 또한 수신 데이터가 변경될 때마다 출력 결과가 즉시 반영되도록 구현하였다.

· 회로 구조

회로는 UART 수신부, 데이터 처리부, LED 제어부로 구성된다.

· 핵심 코드 및 설명

- Top Moudle

```
1  module top(
2      input clk,
3      input rx,
4      output tx,
5      output led
6  );
```

- LED상태 적양용 레지스터

```
1  reg led_reg;
```

```
1  assign led = led_reg;
```

LED 상태 저장용 레지스터 추가 및 출력을 연결한다.

· 핵심 코드 및 설명

```
1  module top(
2      input clk,
3      input rx,
4      output tx,
5      output led
6  );
```

Top Module(최상위 모듈)을 수정한다.

```
1  reg led_reg;
```

```
1  assign led = led_reg;
```

LED 상태 저장용 레지스터 추가 및 출력을 연결한다.

```
wire [7:0] rx_data;
wire rx_done;
```

LED 상태 저장용 레지스터 추가 및 출력을 연결한다.

```
always @(posedge clk)
begin
    if (rx_done)
    begin
        case (rx_data)

            8'h30:
                led_reg <= 1'b0;

            8'h31:
                led_reg <= 1'b1;

            8'h32:
                led_reg <= ~led_reg;

            default:
                led_reg <= led_reg;

        endcase
    end
end
```

UART를 통해 수신된 문자 데이터를 확인하여 LED의 상태를 제어한다. 수신 완료 신호가 발생하면 입력된 데이터를 검사하고, 문자 '0'이 입력되면 LED를 OFF, 문자 '1'이 입력되면 ON, 문자 '2'가 입력되면 현재 LED 상태를 반전(Toggle)하도록 동작한다.

· 검증 결과

시뮬레이션 결과 UART로 전송된 데이터가 정상적으로 수신되는 것을 확인하였다. 또한 수신 완료 신호 발생 후 LED 출력이 설계한 조건에 맞게 변경되는 것을 확인하였다. FPGA 보드 실험에서도 송신 측에서 전송한 값에 따라 LED가 정상적으로 점등 및 소등되었으며, 연속적인 데이터 전송 환경에서도 수신 오류 없이 안정적인 동작을 수행하였다. 이를 통해 UART 수신 회로와 LED 제어 회로가 정상적으로 동작함을 검증하였다.

· 문제 해결 과정

실험 과정에서 UART 수신 시점과 LED 갱신 시점이 맞지 않아 출력이 즉시 반영되지 않는 문제가 발생할 수 있었다. 이를 해결하기 위해 수신 완료 신호(rx_done) 발생 시에만 데이터를 저장하도록 수정하여 잘못된 데이터가 LED에 출력되지 않도록 하였다. 또한 UART 전송 속도와 FPGA 시스템 클록 간의 차이로 인해 발생할 수 있는 수신 오류를 방지하기 위해 비트 카운터와 샘플링 타이밍을 점검하였다. 입력 데이터가 변경될 때마다 LED 상태가 안정적으로 갱신되는 것을 확인하여 최종적으로 신뢰성 있는 UART-LED 제어 시스템을 구현할 수 있었다.

7. 최종 고찰

과제 5부터 과제 7까지의 실험을 통해 UART 통신의 기본 원리와 이를 활용한 주변 장치 제어 방법을 단계적으로 학습할 수 있었다. 과제 5에서는 PC와 MCU 간의 UART 송수신을 수행하며 문자 데이터가 ASCII 코드 형태로 처리되는 과정을 확인하였다. 이를 통해 직렬 통신의 데이터 전송 방식과 UART 라이브러리 함수의 사용법을 이해할 수 있었다.

과제 6에서는 UART로 수신한 데이터를 이용하여 LED를 제어함으로써 단순한 데이터 송수신을 넘어 수신된 정보를 실제 하드웨어 제어에 활용하는 방법을 학습하였다. 특히 문자 데이터와 ASCII 코드의 차이를 이해하고 조건문을 통해 원하는 기능을 구현하는 과정에서 임베디드 소프트웨어 설계 능력을 향상

시킬 수 있었다.

과제 7에서는 UART 수신 기능을 FPGA에 적용하여 수신된 데이터를 LED 출력으로 연결함으로써 하드웨어 기반 통신 시스템을 구현하였다. 이 과정을 통해 UART 수신기, 데이터 저장 레지스터, 출력 제어 로직이 서로 연계되어 동작하는 구조를 이해할 수 있었으며, FPGA에서의 디지털 회로 설계 및 검증 방법에 대한 이해를 높일 수 있었다.

전체적으로 이번 실험을 통해 UART 통신이 MCU와 FPGA를 비롯한 다양한 디지털 시스템 간의 데이터 교환에 매우 중요한 역할을 수행한다는 것을 확인할 수 있었다. 또한, 데이터 수신 시점의 동기화, ASCII 데이터 처리, 입출력 장치 제어 등의 요소가 시스템의 정상 동작에 큰 영향을 미친다는 점을 알게 되었다. 향후에는 단순 LED 제어를 넘어 센서 데이터 수집, MCU-FPGA 간 통신, SPI·I2C 등 다른 통신 프로토콜과의 연계를 통해 보다 복잡한 임베디드 시스템을 구현할 수 있을 것으로 기대된다.

8. 출처

[1] STMicroelectronics, STM32 Nucleo-64 Boards User Manual, STMicroelectronics.

[2] Jonathan Valvano, Embedded Systems: Introduction to ARM Cortex-M Microcontrollers, CreateSpace Independent Publishing Platform.

9. Git-Hub 주소 및 최종 커밋

https://github.com/UnnamedDays/ECE-II-Project

905c6f6